



Green University of Bangladesh
Department of Computer Science and Engineering (CSE)
Faculty of Sciences and Engineering
Semester: (Spring, Year:2025), B.Sc. in CSE (Day)

Lab Report 04
Genetic Algorithm

CourseTitle: Artificial Intelligence Lab
Course Code: CSE-316 Section:221-14

Student Details

Name	ID
Abu Bakkar Siddik	221902265

Lab Date : 15-04-2025
Submission Date : 03-01-2025
Course Teacher's Name : Md. Sabbir Hosen Mamun

[For Teachers use only: **Don't Write Anything inside this box**]

Lab Report Status

Marks:
Comments:.....

Signature:.....
Date:.....

Lab Report Name: Solving the N-Queens Problem Using Genetic Algorithms.

1. Introduction

The N-Queens problem is a well-known combinatorial puzzle in computer science and artificial intelligence. The objective is to position N queens on an $N \times N$ chessboard in such a way that no two queens threaten each other. In other words, no two queens should occupy the same row, column, or diagonal. As the value of N increases, the complexity of solving the problem also grows. Genetic Algorithms (GAs), which mimic the principles of natural selection and evolution, provide a powerful heuristic approach to finding near-optimal solutions. They work by evolving a population of potential solutions through processes such as selection, crossover, and mutation.

2. Objective

The main goal of this lab experiment is to:

- Implement the Genetic Algorithm approach to address the N-Queens challenge.
- Gain insight into the roles of chromosome encoding, fitness assessment, selection mechanisms, crossover operations, and mutation within evolutionary algorithms.
- Analyze the effectiveness of Genetic Algorithms in handling constraint satisfaction problems.

3. Problem Statement

Develop and execute a Python program that applies a Genetic Algorithm to solve the N-Queens problem. The program should:

- Encode potential solutions as chromosomes.
- Calculate fitness by assessing the number of queen conflicts.
- Implement genetic operations such as selection, crossover, and mutation.
- Iterate through generations until a solution with optimal fitness (no conflicts) is discovered.

4. Procedure

Design a Genetic Algorithm to solve the N-Queens problem by representing each solution as a chromosome (a list of integers where each value indicates the queen's row in a specific column), initializing a random population, evaluating fitness based on non-attacking queen pairs, selecting parents using roulette wheel selection, applying one-point crossover and occasional mutation to introduce variation, and repeating the process until a conflict-free solution is found—then output the optimal arrangement and its corresponding chessboard configuration.

Chromosome Representation: Solutions are encoded as lists of integers, each of length N . The number at each index indicates the row position of a queen in the corresponding column.

Initialization: An initial pool of random chromosomes is created to form the starting population.

Fitness Function:

Fitness is measured by counting how many pairs of queens do not attack each other. Conflicts along rows and diagonals reduce the fitness score.

Selection: Parents are selected using a probability-based approach, such as roulette wheel selection, giving preference to fitter individuals.

Crossover: A single crossover point is selected to exchange genetic material between two parent chromosomes, producing a new child chromosome.

Mutation: To maintain variety in the population, a low-probability mutation (e.g., 3%) is applied, where a queen's position is changed to a different row.

Termination: The process repeats across generations until a solution with perfect fitness—meaning no queens are in conflict—is found.

Output: Once a valid solution is identified, display both the chromosome and a formatted chessboard layout showing where the queens are placed.

5. Implementation

```
import random

def random_chromosome(size):
    return [random.randint(1, size) for _ in range(size)]

def fitness(chromosome):
    horizontal_collisions = sum([chromosome.count(queen)-1 for queen in chromosome]) / 2
    n = len(chromosome)
    left_diagonal = [0] * (2 * n)
    right_diagonal = [0] * (2 * n)

    for i in range(n):
        left_diagonal[i + chromosome[i] - 1] += 1
        right_diagonal[n - i + chromosome[i] - 2] += 1

    diagonal_collisions = 0
    for i in range(2 * n - 1):
        if left_diagonal[i] > 1:
```

```

        diagonal_collisions += (left_diagonal[i] - 1)
        if right_diagonal[i] > 1:
            diagonal_collisions += (right_diagonal[i] - 1)

    return int(maxFitness - (horizontal_collisions + diagonal_collisions))

def probability(chromosome, fitness_func):
    return fitness_func(chromosome) / maxFitness

def random_pick(population, probabilities):
    r = random.uniform(0, sum(probabilities))
    upto = 0
    for c, w in zip(population, probabilities):
        if upto + w >= r:
            return c
        upto += w
    return population[0]

def reproduce(x, y):
    n = len(x)
    c = random.randint(0, n - 1)
    return x[0:c] + y[c:n]

def mutate(x):
    n = len(x)
    c = random.randint(0, n - 1)
    m = random.randint(1, n)
    x[c] = m
    return x

def genetic_queen(population, fitness_func):
    mutation_probability = 0.03
    new_population = []
    probabilities = [probability(n, fitness_func) for n in population]

    for i in range(len(population)):
        x = random_pick(population, probabilities)
        y = random_pick(population, probabilities)
        child = reproduce(x, y)

        if random.random() < mutation_probability:
            child = mutate(child)

        print_chromosome(child)
        new_population.append(child)

        if fitness_func(child) == maxFitness:
            break

    return new_population

```

```

def print_chromosome(chrom):
    print("Chromosome = {}, Fitness = {}".format(str(chrom), fitness(chrom)))

def print_board(board):
    for row in board:
        print(" ".join(row))

if __name__ == "__main__":
    nq = int(input("Enter Number of Queens: "))
    maxFitness = (nq * (nq - 1)) / 2
    population = [random_chromosome(nq) for _ in range(100)]

    generation = 1

    while not maxFitness in [fitness(chrom) for chrom in population]:
        print("=== Generation {} ===".format(generation))
        population = genetic_queen(population, fitness)
        print("Maximum Fitness = {}".format(max([fitness(n) for n in population])))
        generation += 1

    print("Solved in Generation {}".format(generation - 1))

    for chrom in population:
        if fitness(chrom) == maxFitness:
            print("\nOne of the solutions: ")
            print_chromosome(chrom)
            chrom_out = chrom
            Break

    board = [["x"] * nq for _ in range(nq)]
    for i in range(nq):
        board[nq - chrom_out[i]][i] = "Q"

    print()
    print_board(board)

```

6. Output

```
Enter Number of Queens: 5
--- Generation 1 ---
Chromosome - [2, 1, 5, 2, 1], Fitness - 6
Chromosome - [2, 3, 1, 1, 2], Fitness - 5
Chromosome - [4, 4, 4, 1, 5], Fitness - 6
Chromosome - [1, 3, 2, 2, 5], Fitness - 7
Chromosome - [2, 4, 4, 2, 3], Fitness - 5
Chromosome - [2, 2, 4, 5, 1], Fitness - 7
Chromosome - [2, 4, 4, 2, 2], Fitness - 3
Chromosome - [4, 1, 3, 3, 1], Fitness - 6
Chromosome - [2, 1, 3, 2, 1], Fitness - 5
Chromosome - [3, 4, 5, 1, 1], Fitness - 6
Chromosome - [5, 1, 2, 3, 5], Fitness - 7
Chromosome - [1, 3, 3, 3, 2], Fitness - 5
Chromosome - [2, 3, 3, 5, 1], Fitness - 6
Chromosome - [5, 5, 3, 4, 1], Fitness - 6
Chromosome - [1, 5, 1, 1, 1], Fitness - 4
Chromosome - [5, 5, 3, 3, 1], Fitness - 5
Chromosome - [4, 4, 1, 2, 2], Fitness - 6
Chromosome - [4, 3, 2, 4, 3], Fitness - 5
Chromosome - [1, 3, 3, 4, 2], Fitness - 7
Chromosome - [3, 5, 2, 4, 3], Fitness - 8
Chromosome - [3, 1, 2, 3, 1], Fitness - 6
Chromosome - [5, 5, 2, 4, 1], Fitness - 8
Chromosome - [5, 3, 3, 3, 4], Fitness - 5
Chromosome - [2, 5, 4, 2, 2], Fitness - 4
Chromosome - [5, 5, 5, 2, 4], Fitness - 6
Chromosome - [4, 3, 2, 4, 5], Fitness - 6
Chromosome - [4, 1, 3, 5, 2], Fitness - 10
Maximum Fitness - 10
Solved in Generation 1!

One of the solutions:
Chromosome - [4, 1, 3, 5, 2], Fitness - 10

x x x Q x
Q x x x x
x x Q x x
x x x x Q
x Q x x x

--- Code Execution Successful ---
```

7. Conclusion

This experiment effectively showcased the application of Genetic Algorithms in solving the N-Queens problem with efficiency. By emulating the principles of natural evolution, the algorithm gradually enhances potential solutions across generations. Unlike exhaustive search methods, the GA offers strong performance even as the value of N increases. However, because it relies

on randomness, the time to reach a solution can differ from run to run. In summary, this technique is a flexible, scalable, and reliable approach for tackling constraint-based problems.

GitHub Link: <https://github.com/ShawonTech/LabReport05./tree/main>